

CW1+2 Coursework Project –Task Sheet

(20% of the module mark)

➤ **Overview**

The Coursework Project is a two-part assignment in which you compute a collision-free synchronized schedule for two hospital courier robots on a rectangular grid map. Each part is assessed independently via a 45 minutes in-class test, each weighted 10% of the overall module mark. In Part 1, you will be limited to an array-first design and simple linked-list support structures. Part 2 changes the focus to a graph model: the same problem is re-expressed as a search on a configuration graph, solved with breadth-first search.

➤ **Components**

#	Component	Mode	Module Weight	Learning Outcomes	Submission
CW1.1	Part 1 Take-home Assignment	Group	Prerequisite for In-Class Test 1	A, B, D	Week 6
CW1.2	In-class Test 1	Individual	10%	A, B, D	Week 7
CW2.1	Part 2 Take-home Assignment	Group	Prerequisite for In-Class Test 2	ALL	Week 11
CW1.2	In-class Test 2	Individual	10%	ALL	Week 12

➤ **Learning Outcomes**

The Coursework Project covers the following module learning outcomes:

- A. Identify and describe the principles of common data structures (e.g., arrays, lists, stacks, queues, trees, and graphs) and demonstrate their design and implementation in a programming language.
- B. Explain and illustrate the use of algorithms to create/access/manage data structures and apply this knowledge to solve specific problems effectively.
- C. Conduct basic analyses on data structures related algorithm performance with reference to time and space complexity, and interpret the results.
- D. Select, implement, and justify the use of the most appropriate data structures for given problems, considering the efficiency of different algorithms.

➤ Collaboration and Academic Integrity

This module recognises that interactions with classmates and others can facilitate mastery of the module's materials. However, there remains a line between enlisting the help of another and submitting the work of another. This statement characterises both sides of that line.

The essence of all work submitted to this module for credit must be **entirely your own work or your group's own work** depends on the assessment mode. For group work, collaboration on completing them with any non-group member is prohibited except to the extent that you may ask classmates and others for some general hints, as opposed to sharing/receiving specific solutions. For group work, **all group members are equally and collectively responsible for the entire submission. All group members are held jointly accountable for the integrity of the entire submission.** Each group member acts as a guarantor of the group's academic integrity and must actively monitor contributions and violations of academic integrity. All members must confirm their submission and sign the submission cover sheet. If any member violates the university academic integrity policy without the group's immediate reporting, **all members share the responsibilities and will face penalties.**

Plagiarism, copying, collusion, or dishonest use of data will be penalised. Disciplinary actions and mark penalties ranging from capped scores to an award of zero can be applied.

Please refer to the [University's Academic Integrity Policy on e-bridge](#) for guidance. You may also contact module leaders or examination officers if you have any confusion relating to academic integrity.

➤ Problem specification

A hospital deploys two courier robots. Robot A carries medication and Robot B carries lab samples. Each robot has a start-point (denoted S_A for Robot A, S_B for Robot B) and a designated *target* (denoted T_A for Robot A, T_B for Robot B) to deliver their product to. The hospital map is represented as a rectangular grid containing walls and free cells, where the start-points and targets of both robots are free cells, see Figure 1 for an example. The goal is for the robots to reach their targets as quickly as possible. We consider a discrete-time setting. At time $t=0$, Robot A is in cell S_A , and Robot B in cell S_B . At each step of time (called a *tick*), both robots choose either to move into an adjacent cell (in North, East, South, or West directions) in the map, or to remain in their current position. As such, at each tick, both robots choose one move from $\{N, E, S, W, WAIT\}$.

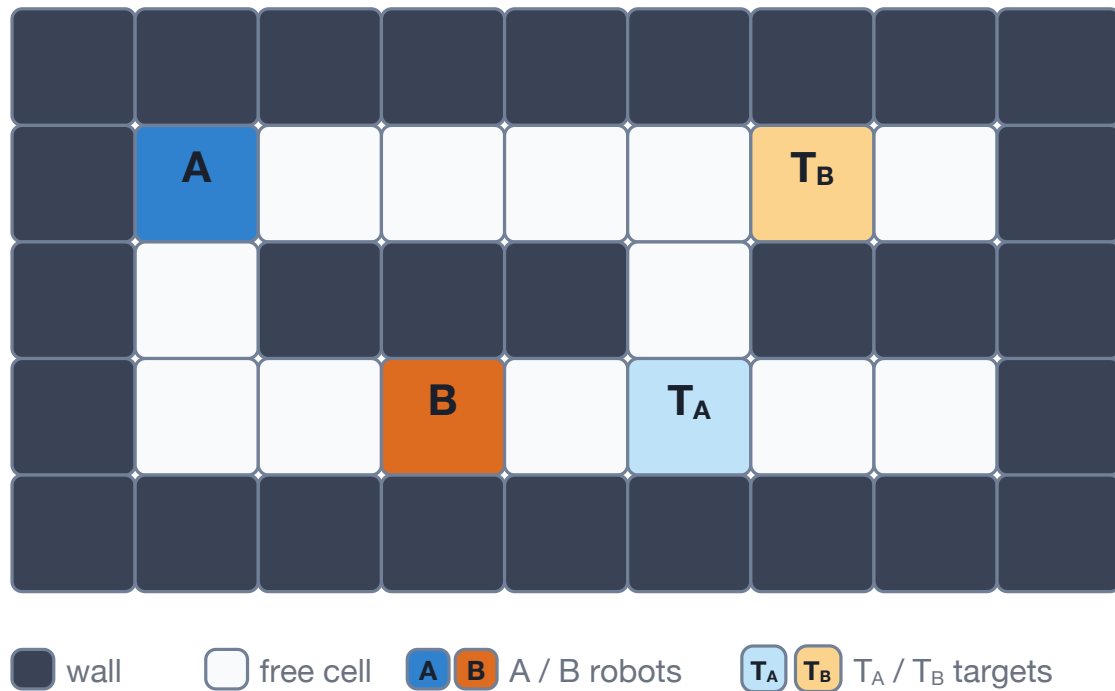


Figure 1: An example of a hospital map

Moves made by the robots must satisfy the following conditions:

- Robots may not move into walls or outside the map. For example, in Figure 1, Robot A can only move East, South, or WAIT, since otherwise it would move into a wall. Similarly Robot B can only move East, West, or WAIT.
- After a tick, robots must occupy different cells. For example, in Figure 1, we could not have Robot A moving S-S (South then South) and Robot B moving W-W, since then both robots would occupy the same cell (the free cell in the bottom-left corner).
- Robots may not swap cells in the same tick. For example, in Figure 1, we could not have Robot A moving S-S-E and Robot B moving W-WAIT-W, since the robots would swap cells on the last tick.

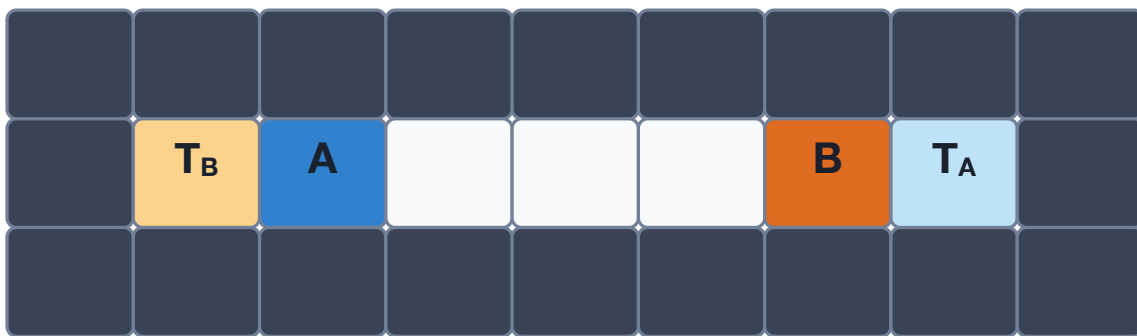
A pair of moves satisfying the above conditions is called *legal*.

A *synchronised schedule* is a sequence of moves $(M_A(0), M_B(0)), (M_A(1), M_B(1)), \dots$, where $(M_A(t), M_B(t))$ is the pair of moves made by the robots in the tick from time t to time $t+1$, such that:

- Starting at S_A and applying the moves $M_A(0), M_A(1), \dots$, Robot A reaches its target T_A ;
- Starting at S_B and applying the moves $M_B(0), M_B(1), \dots$, Robot B reaches its target T_B ;
- Every pair of moves $(M_A(t), M_B(t))$ is legal (in the above sense).

The *makespan* of a synchronised schedule is the number of ticks in the schedule (the length of the sequence). A synchronised schedule is said to be *optimal* if it minimises the makespan; that is, there are no synchronised schedules with a smaller makespan. For example, consider the map on Figure 1, where the cells A/B represent the starting points of the two robots. Then (E, E), (E, E), (E, N), (WAIT, N), (E, E), (S, WAIT), (S, WAIT) is a synchronised schedule with makespan 7. However, it is not optimal, since there exists a synchronised schedule with makespan 6 given by (S,E), (S,E), (E,N), (E,N), (E,E), (E,WAIT). This latter synchronised schedule is optimal (for example, note that it will always take Robot A at least six moves to reach its target). However, it is not the only optimal synchronised schedule. Indeed, the WAIT move for Robot B could be inserted at any point in its move sequence E-E-N-N-E while leaving Robot A's moves unchanged (S-S-E-E-E-E).

Synchronised schedules do not always exist. For example, for the map in Figure 2, it is impossible for Robot A to reach its target T_A and Robot B to reach its target T_B without either both robots occupying the same cell or swapping cells in the same tick.



The aim of this coursework is as followed. Given an input hospital map with starting cells S_A and S_B for Robots A and B, and target cells T_A and T_B :

1. Determine if a synchronised schedule exists.
2. If so, compute an optimal synchronised schedule.

In Part 1, you will solve this problem using arrays using a brute-force approach, by calculating all possible pairs of legal moves from any given positioning of Robots A and B in the hospital map. In Part 2, you will represent the problem as a graph, where vertices are (legal) pairs of Robot positions, and apply Depth First Search (DFS) techniques to compute an optimal synchronised schedule.